



Benchmarking Models on Edge Hardware

Measuring performance, efficiency, and accuracy to optimize ML models for resource-constrained environments

Why Benchmark?

Resource Constraints

Edge devices operate with **limited compute, memory, and power** resources, creating unique optimization challenges

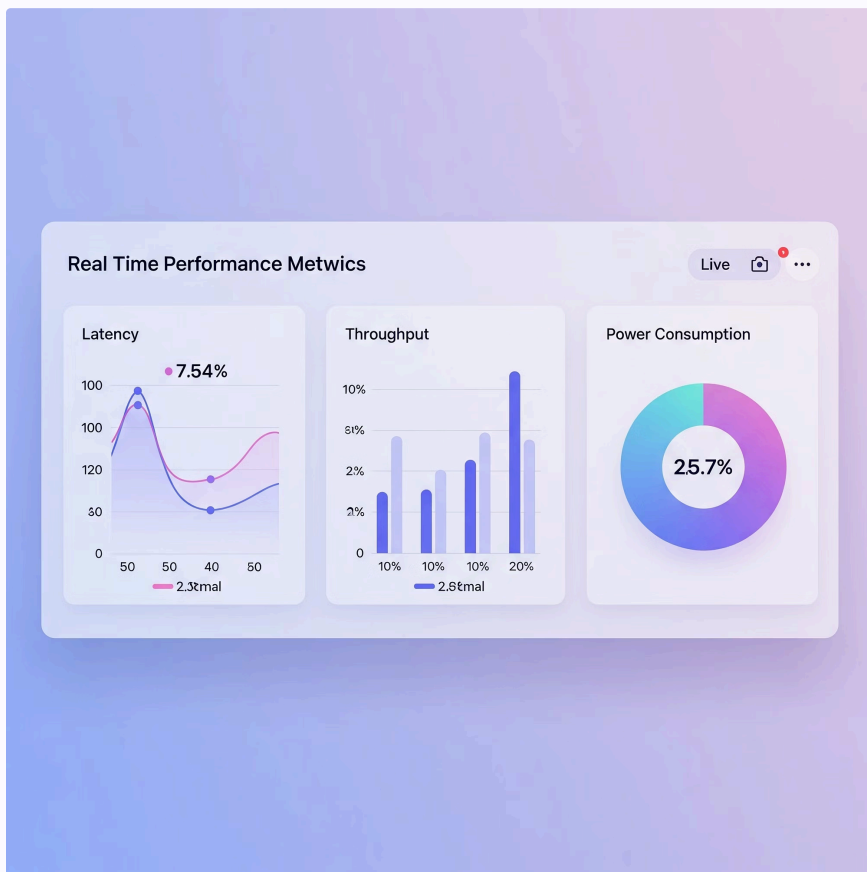
Performance Variability

Different models behave differently under constraints, requiring empirical testing on target hardware

Deployment Confidence

Benchmarking ensures you meet **latency & throughput SLAs** while balancing the critical **accuracy vs. efficiency** trade-off

Key Metrics to Measure



Performance

Latency (ms per inference, p50/p95)

Throughput
(inferences/sec, FPS for vision)

Quality

Accuracy (top-1/F1/precision-recall)

Memory footprint (RAM usage, model size)

Efficiency

Power draw (Watt consumption, battery drain)

Benchmarking Workflow

Select candidate models

Choose architectures appropriate for edge (e.g., MobileNet, ResNet, YOLO variants)

Convert to edge-optimized format

Transform models to deployment-ready formats (TFLite, TensorRT, ONNX)

Deploy to target hardware

Install on actual edge devices (Jetson, Raspberry Pi, MCUs)

Run standardized benchmark dataset

Execute inference on consistent test data that represents real-world conditions

Collect comprehensive metrics

Measure latency, accuracy, memory usage, and energy consumption

Compare against baseline

Evaluate results against requirements and alternative models

Tools & Frameworks



TensorFlow Lite

The `benchmark_model` tool provides standardized metrics for TFLite models



ONNX Runtime

Performance tests for cross-platform model optimization and benchmarking



MLPerf Tiny/Inference

Industry-standard benchmarks for consistent cross-model comparison



PyTorch Mobile

Profiling tools for PyTorch-based edge deployments



Custom Profiling

Using `time.perf_counter`, `torch.cuda.synchronize()` for specialized metrics



Example: TFLite Benchmark on Raspberry Pi



```
./benchmark_model \
--graph=mobilenet_v2.tflite \
--input_layer="input" \
--input_layer_shape="1,224,224,3" \
--num_threads=4
```

Output Metrics:

- Inference latency (ms)
- Memory usage (MB)
- CPU utilization (%)
- Initialization time
- Per-layer performance breakdowns

Example: TensorRT on Jetson

```
trtexec --onnx=resnet18.onnx \  
--fp16 \  
--shapes=input:1x3x224x224
```

TensorRT optimizes models for NVIDIA hardware through kernel fusion, precision calibration, and memory planning.

3.2ms

FP32 Latency

Baseline precision

1.8ms

FP16 Latency

Mixed precision

0.9ms

INT8 Latency

Quantized precision



Edge Device Categories



Microcontrollers (TinyML)

Ultra-low power devices running keyword spotting, anomaly detection

Models: tinyML, MicroNets, ProxylessNAS



Raspberry Pi-class

Single-board computers for MobileNet, EfficientNet-Lite, small NLP

Models: MobileNetV2, EfficientDet-Lite



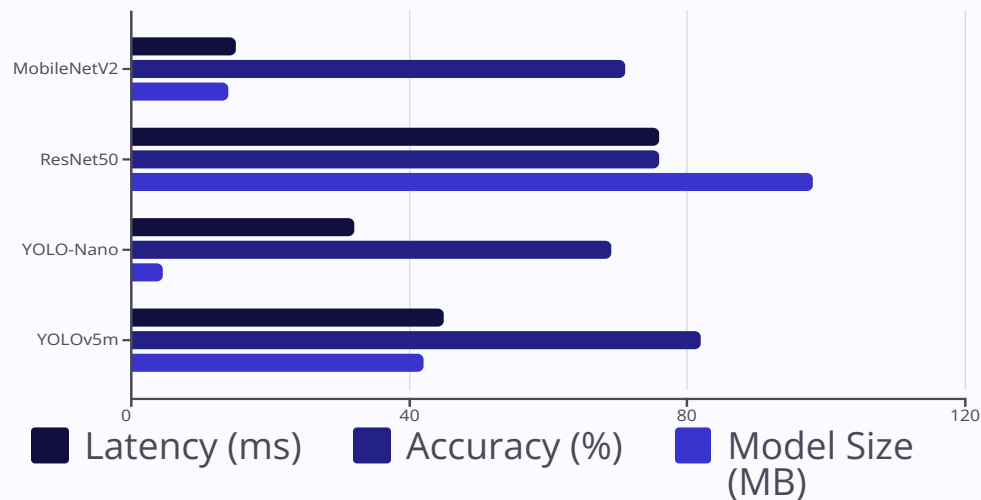
Jetson/Edge GPUs

GPU-accelerated devices for YOLO, ResNet, Transformers-lite

Models: YOLOv5, ResNet18, DistilBERT

The benchmark approach must match **device class + task SLA** to be meaningful

Trade-Off Analysis



Key Trade-off Patterns:

- MobileNet → Fast, lightweight, slight accuracy drop
- ResNet → Better accuracy, heavier compute requirements
- YOLO-Nano vs YOLOv5 → Speed vs detection precision

□ Benchmarking provides quantitative data to make informed trade-off decisions based on your specific application requirements

Power & Thermal Testing



Measurement Tools:

- USB inline power meters
- Jetson tegrastats utility
- Thermal imaging cameras
- System power monitoring APIs

Key Metrics:

- Watts under various workloads
- Thermal throttling thresholds
- **Accuracy-per-Watt** efficiency
- Battery life estimation

Critical for battery-powered and remote deployments where power efficiency determines system viability

Best Practices for Edge Benchmarking

1

Use Real Hardware

Always benchmark on **actual target devices**, not just simulators or development workstations

2

Representative Datasets

Test with data that reflects **real-world edge conditions** including challenging lighting, noisy audio, or variable inputs

3

Measure Complete Latency

Include both **cold-start initialization** and **steady-state inference** latency in your metrics

4

Batch Size Variations

Test with **different batch sizes** and quantization levels to find optimal throughput balance

5

Automate Testing

Integrate benchmarking into your **CI/CD pipeline** to catch performance regressions early



Key Takeaways

Effective edge AI requires matching model capabilities to device constraints

Essential Edge Benchmarking Elements:

Comprehensive metrics: latency, throughput, accuracy, memory, power

Right tools for each platform: TFLite Benchmark, TensorRT trtexec, MLPerf

Real-world testing ensures deployable, reliable edge AI systems

Remember: **What you can't measure, you can't optimize**